

OpenAPI do zero

Guia didático para especificar APIs HTTP em ambiente AWS

1. Objetivo deste material

Este material ensina:

- o que é OpenAPI;
- qual problema ele resolve;
- a diferença entre OpenAPI, Swagger e API Gateway;
- como escrever uma especificação;
- como documentar endpoints;
- como definir parâmetros obrigatórios e opcionais;
- como descrever o corpo da resposta;
- como definir erros;
- como documentar autenticação;
- como reutilizar modelos;
- como usar o contrato no Amazon API Gateway;
- como apresentar a mesma documentação para negócio e tecnologia.

Nos exemplos, será utilizada a versão **OpenAPI 3.0.4**. Embora a versão mais recente publicada da especificação seja a 3.2.0, o Amazon API Gateway documenta a importação de arquivos OpenAPI 3.0. Portanto, para um projeto que será integrado diretamente ao API Gateway, a família 3.0 é uma escolha prática e compatível.

2. O que é uma API?

Uma API é um contrato de comunicação entre sistemas.

Considere um sistema consumidor que deseja consultar um pedido:

```
GET /v1/pedidos/PED-12345
```

A API pode responder:

```
{
  "pedidoId": "PED-12345",
  "status": "EM_PROCESSAMENTO",
  "dataCriacao": "2026-07-15T14:30:00-03:00"
}
```

Para que essa integração funcione, o consumidor precisa saber:

- qual endereço chamar;
- qual método HTTP utilizar;
- quais parâmetros enviar;
- quais parâmetros são obrigatórios;
- qual autenticação utilizar;
- qual estrutura será retornada;
- quais valores cada campo pode assumir;
- quais erros podem ocorrer.

Essas informações formam o **contrato da API**.

3. O que é OpenAPI?

OpenAPI é um padrão para descrever o contrato de APIs HTTP de maneira estruturada, independente da linguagem de programação usada na implementação.

A especificação pode ser lida:

- por pessoas;
- por ferramentas de documentação;
- por geradores de código;
- por ferramentas de testes;
- por validadores;
- por gateways de API.

Uma especificação OpenAPI permite entender as capacidades de uma API sem precisar examinar o código-fonte ou observar seu tráfego de rede.

Um arquivo OpenAPI normalmente descreve:

```
Nome e versão da API
↓
Servidores e ambientes
↓
Endpoints disponíveis
↓
Métodos HTTP
↓
Parâmetros de entrada
↓
Bodies de requisição
↓
Bodies de resposta
↓
Erros
↓
```

Autenticação



Modelos de dados

O que OpenAPI não é

OpenAPI não é:

- a implementação da API;
- um servidor HTTP;
- um banco de dados;
- o Amazon API Gateway;
- uma função Lambda;
- um framework como FastAPI ou Spring Boot;
- uma interface gráfica.

Ele é a **descrição formal do contrato**.

4. Diferença entre OpenAPI e Swagger

Esses nomes são frequentemente confundidos.

OpenAPI

É o padrão ou linguagem utilizada para descrever a API.

Exemplo:

```
openapi: 3.0.4
info:
  title: API de Pedidos
  version: 1.0.0
```

Swagger

Swagger é um conjunto de ferramentas que trabalha com arquivos OpenAPI.

Exemplos:

Ferramenta	Função
Swagger Editor	Criar e validar o arquivo OpenAPI
Swagger UI	Gerar uma página interativa da API
Swagger Codegen	Gerar clientes e estruturas de código

Ferramenta	Função
Swagger CLI	Validar ou processar documentos

O nome antigo da especificação era Swagger Specification. Posteriormente, a especificação passou a se chamar OpenAPI Specification. Atualmente, OpenAPI é o padrão, enquanto Swagger identifica principalmente o ecossistema de ferramentas.

Uma forma simples de memorizar:

OpenAPI = o padrão e o arquivo de contrato
 Swagger = ferramentas que trabalham com esse contrato

5. Onde o Amazon API Gateway entra?

O API Gateway é um serviço da AWS que recebe requisições HTTP e as encaminha para uma integração, como:

- AWS Lambda;
- outro serviço HTTP;
- aplicações em ECS;
- aplicações em EKS;
- serviços internos acessíveis por VPC Link.

Você pode escrever uma especificação OpenAPI e importá-la no API Gateway para criar a estrutura da API. O API Gateway também possui extensões próprias, identificadas normalmente pelo prefixo:

x-amazon-apigateway-

Essas extensões permitem definir detalhes específicos da AWS, como a integração de um endpoint com uma função Lambda. O API Gateway permite criar HTTP APIs a partir de arquivos OpenAPI 3.0, embora nem todos os recursos do padrão sejam necessariamente suportados pelo serviço.

A relação é:

```

Arquivo OpenAPI
  ↓
Amazon API Gateway
  ↓
Lambda, ECS, EKS ou outro backend
  
```

6. Formato do arquivo OpenAPI

O documento pode ser escrito em:

- YAML;
- JSON.

Os dois formatos representam a mesma estrutura. A especificação define o documento como um objeto JSON, que pode ser representado em JSON ou YAML. Os nomes dos campos são sensíveis a letras maiúsculas e minúsculas.

Exemplo em YAML

```
openapi: 3.0.4

info:
  title: API de Pedidos
  version: 1.0.0
```

O mesmo exemplo em JSON

```
{
  "openapi": "3.0.4",
  "info": {
    "title": "API de Pedidos",
    "version": "1.0.0"
  }
}
```

YAML costuma ser mais confortável para escrita e revisão humana.

7. Noções básicas de YAML

7.1 Chave e valor

```
title: API de Pedidos
```

- `title` é a chave;
- `API de Pedidos` é o valor.

7.2 Objetos são representados por indentação

```
info:
  title: API de Pedidos
  version: 1.0.0
```

`title` e `version` pertencem ao objeto `info`.

7.3 Listas usam hífen

```
servers:
- url: https://api.exemplo.com.br
- url: https://sandbox-api.exemplo.com.br
```

7.4 Texto com várias linhas

```
description: |
  Esta API permite consultar pedidos.

  Somente consumidores autorizados podem realizar consultas.
```

O símbolo `|` indica um texto com múltiplas linhas.

7.5 Comentários

```
openapi: 3.0.4 # Versão da especificação OpenAPI
```

Tudo o que aparece depois de `#` é um comentário.

7.6 Cuidados

Use espaços para indentação. Evite tabulações.

Estes dois níveis são diferentes:

```
info:
  title: API de Pedidos
```

```
info:
title: API de Pedidos
```

No segundo exemplo, `title` não está dentro de `info`.

8. Estrutura mínima de um OpenAPI

Um documento mínimo pode ser:

```
openapi: 3.0.4

info:
  title: API de Pedidos
  version: 1.0.0

paths: {}
```

Os principais campos obrigatórios no nível raiz são:

- `openapi`;
- `info`;
- `paths`.

Dentro de `info`, os campos obrigatórios são `title` e `version`.

Significado

```
openapi: 3.0.4
```

Informa qual versão do padrão foi usada para escrever o arquivo.

```
info:
```

Contém informações gerais sobre a API.

```
paths:
```

Contém os endpoints e operações disponíveis.

9. Estrutura completa do nível principal

Um documento mais completo possui a seguinte estrutura:

```
openapi: 3.0.4

info:
```

```

title: API de Pedidos
description: API utilizada para consultar pedidos.
version: 1.0.0

servers:
  - url: https://api.exemplo.com.br/v1

tags:
  - name: Pedidos
    description: Operações relacionadas a pedidos.

paths:
  /pedidos/{pedidoId}:
    get:
      summary: Consultar pedido
      responses:
        "200":
          description: Consulta realizada com sucesso.

components:
  schemas: {}

security: []

externalDocs:
  description: Documentação funcional
  url: https://documentacao.exemplo.com.br

```

Campos do nível principal

Campo	Obrigatório	Significado
<code>openapi</code>	Sim	Versão da especificação utilizada
<code>info</code>	Sim	Informações gerais da API
<code>servers</code>	Não	Endereços dos ambientes
<code>paths</code>	Sim	Endpoints e métodos disponíveis
<code>components</code>	Não	Componentes reutilizáveis
<code>security</code>	Não	Autenticação padrão da API
<code>tags</code>	Não	Agrupamento dos endpoints
<code>externalDocs</code>	Não	Link para documentação complementar

10. O objeto `info`

Exemplo:

```
info:
  title: API de Consulta de Pedidos
  description: |
    API responsável por disponibilizar informações de pedidos.

    A API é consumida pelo portal de atendimento e pelo aplicativo.
  version: 1.0.0

  contact:
    name: Squad de Pedidos
    email: squad-pedidos@exemplo.com.br

  license:
    name: Uso interno

  termsOfService: https://exemplo.com.br/termos
```

Campos

`title`

Nome da API.

```
title: API de Consulta de Pedidos
```

Deve ser compreensível para negócio e tecnologia.

Evite:

```
title: XPTO-API-01
```

Prefira:

```
title: API de Consulta de Pedidos
```

`description`

Descrição detalhada da API.

```
description: |  
  Permite consultar pedidos realizados pelos clientes.  
  
  Somente sistemas autorizados podem acessar a operação.
```

Esse campo pode conter:

- objetivo funcional;
- público consumidor;
- limitações;
- regras gerais;
- observações de segurança.

version

Versão do contrato da API.

```
version: 1.0.0
```

Não confunda:

```
openapi: 3.0.4
```

com:

```
info:  
  version: 1.0.0
```

A primeira é a versão do padrão OpenAPI.

A segunda é a versão da sua API ou do seu contrato. A própria documentação do padrão destaca que esses valores têm significados diferentes.

contact

Identifica a equipe responsável.

```
contact:  
  name: Squad de Pedidos  
  email: squad-pedidos@exemplo.com.br
```

license

Informa a licença ou condição de utilização.

```
license:
  name: Uso interno
```

`termsOfService`

Endereço dos termos de uso.

```
termsOfService: https://exemplo.com.br/termos
```

11. O objeto `servers`

O campo `servers` informa os endereços-base da API.

```
servers:
  - url: https://api.exemplo.com.br/v1
    description: Produção

  - url: https://sandbox-api.exemplo.com.br/v1
    description: Ambiente de testes
```

Os caminhos definidos em `paths` são acrescentados ao endereço do servidor.

Exemplo:

```
servers:
  - url: https://api.exemplo.com.br/v1

paths:
  /pedidos/{pedidoId}:
```

URL resultante:

```
https://api.exemplo.com.br/v1/pedidos/{pedidoId}
```

OpenAPI permite declarar múltiplos servidores, por exemplo produção e sandbox. Se `servers` não for informado, o endereço-base padrão é `/`.

Variáveis no servidor

Também é possível usar variáveis:

```
servers:
  - url: https://{ambiente}.api.exemplo.com.br/v1
  variables:
    ambiente:
      default: producao
      enum:
        - producao
        - homologacao
        - desenvolvimento
```

12. O objeto `tags`

Tags agrupam operações relacionadas.

```
tags:
  - name: Pedidos
    description: Consulta e gerenciamento de pedidos.

  - name: Clientes
    description: Operações relacionadas a clientes.
```

Depois, a operação referencia a tag:

```
get:
  tags:
    - Pedidos
```

No Swagger UI, isso normalmente faz com que os endpoints apareçam organizados em grupos.

13. O objeto `paths`

`paths` é o catálogo de endpoints.

```
paths:
  /pedidos:
    get:
      summary: Pesquisar pedidos

  /pedidos/{pedidoId}:
    get:
      summary: Consultar um pedido
```

Cada caminho deve começar com `/`. O caminho é relativo ao endereço informado em `servers`.

Path não é a URL completa

Correto:

```
paths:
  /pedidos/{pedidoId}:
```

Incorreto:

```
paths:
  https://api.exemplo.com.br/v1/pedidos/{pedidoId}:
```

14. Métodos HTTP

Dentro de um caminho, você informa as operações HTTP.

```
paths:
  /pedidos/{pedidoId}:
    get:
      summary: Consultar pedido

    delete:
      summary: Excluir pedido
```

Os métodos mais comuns são:

Método	Utilização comum
<code>get</code>	Consultar recursos
<code>post</code>	Criar um recurso ou executar um processamento
<code>put</code>	Substituir integralmente um recurso
<code>patch</code>	Atualizar parcialmente um recurso
<code>delete</code>	Excluir um recurso
<code>head</code>	Consultar apenas cabeçalhos
<code>options</code>	Consultar opções de comunicação

OpenAPI não define sozinho qual método deve ser utilizado. Essa decisão faz parte do desenho da API.

15. Estrutura de uma operação

Uma operação GET pode ter:

```
get:
  tags:
    - Pedidos

  summary: Consultar pedido

  description: |
    Consulta um pedido utilizando seu identificador.

    O consumidor somente pode acessar pedidos de sua organização.

  operationId: consultarPedido

  parameters: []

  responses: {}

  security: []

  deprecated: false
```

summary

Resumo curto da operação.

```
summary: Consultar pedido
```

É o título que geralmente aparece na interface visual.

description

Explicação detalhada.

```
description: |
  Consulta os dados atuais de um pedido.

  O pedido deve pertencer à organização do consumidor.
```

Use esse campo para registrar:

- objetivo;

- regra de negócio;
- pré-condições;
- comportamento esperado;
- limitações;
- observações importantes.

operationId

Identificador técnico único da operação.

```
operationId: consultarPedido
```

Pode ser usado por ferramentas de geração de código.

Cada operação deve ter um `operationId` diferente:

```
operationId: consultarPedido
operationId: pesquisarPedidos
operationId: consultarHistoricoPedido
```

O padrão recomenda que o identificador seja único em todo o documento.

tags

Grupos aos quais a operação pertence.

```
tags:
  - Pedidos
```

deprecated

Informa se a operação está sendo descontinuada.

```
deprecated: true
```

Isso não remove o endpoint. Apenas informa que ele não deve ser utilizado em novas integrações.

responses

Lista de respostas possíveis.

```
responses:
  "200":
    description: Pedido encontrado.
```

O campo `responses` é obrigatório em cada operação.

16. Parâmetros de entrada

Parâmetros são valores enviados fora do body.

OpenAPI reconhece quatro localizações:

Valor de <code>in</code>	Local
<code>path</code>	Parte do caminho
<code>query</code>	Query string
<code>header</code>	Cabeçalho HTTP
<code>cookie</code>	Cookie

Essas quatro localizações são definidas pelo objeto `Parameter`. Cada parâmetro deve possuir `name` e `in`.

17. Parâmetro de path

Exemplo de chamada:

```
GET /v1/pedidos/PED-12345
```

Nesse caso, `PED-12345` é o valor do parâmetro `pedidoId`.

```
paths:
  /pedidos/{pedidoId}:
    get:
      parameters:
        - name: pedidoId
          in: path
          required: true
          description: Identificador único do pedido.
          schema:
```



```
type: string
example: PED-12345
```

Significado dos campos

name

Nome técnico do parâmetro:

```
name: pedidoId
```

Deve ser igual ao nome utilizado no caminho:

```
/pedidos/{pedidoId}
```

in

Local onde o valor será enviado:

```
in: path
```

required

Informa se o parâmetro é obrigatório:

```
required: true
```

Todo parâmetro de path é obrigatoriamente requerido e deve declarar `required: true`.

description

Explica o significado funcional:

```
description: Identificador único do pedido.
```

schema

Define tipo e regras:

```
schema:
  type: string
```

example

Apresenta um valor de exemplo:

example: PED-12345

18. Parâmetro de query

Exemplo:

GET /v1/pedidos?status=CONCLUIDO&pagina=1

OpenAPI:

```
parameters:
  - name: status
    in: query
    required: false
    description: Filtra pedidos pela situação.
    schema:
      type: string
      enum:
        - RECEBIDO
        - EM_PROCESSAMENTO
        - CONCLUIDO
        - CANCELADO
      example: CONCLUIDO

  - name: pagina
    in: query
    required: false
    description: Número da página.
    schema:
      type: integer
      minimum: 1
      default: 1
      example: 1
```

Parâmetros de query são normalmente usados para:

- filtros;
- paginação;
- ordenação;
- opções da consulta.

19. Parâmetro de header

Exemplo:

```
X-Correlation-Id: 550e8400-e29b-41d4-a716-446655440000
```

OpenAPI:

```
parameters:
  - name: X-Correlation-Id
    in: header
    required: false
    description: Identificador utilizado para rastrear a requisição.
    schema:
      type: string
      format: uuid
```

Headers customizados podem ser utilizados para:

- rastreabilidade;
- idempotência;
- contexto do consumidor;
- versionamento específico;
- controles técnicos.

A autenticação normalmente deve ser representada por `securitySchemes`, em vez de criar manualmente um parâmetro `Authorization`.

20. Campos do objeto `Parameter`

Exemplo completo:

```
- name: status
  in: query
  required: false
  deprecated: false
  description: Situação do pedido.
  schema:
    type: string
    enum:
      - RECEBIDO
      - CONCLUIDO
  example: CONCLUIDO
```

Campo	Significado
<code>name</code>	Nome do parâmetro
<code>in</code>	Local: path, query, header ou cookie
<code>required</code>	Se o envio é obrigatório
<code>description</code>	Explicação funcional
<code>deprecated</code>	Se está sendo descontinuado
<code>schema</code>	Tipo, formato e validações
<code>example</code>	Exemplo de valor
<code>style</code>	Forma de serialização
<code>explode</code>	Como arrays e objetos serão expandidos
<code>allowReserved</code>	Se caracteres reservados podem passar sem codificação adicional

Para a maioria das APIs, os campos mais importantes são:

```
name
in
required
description
schema
example
```

21. GET possui body de entrada?

Para endpoints GET, prefira enviar dados por:

- path;
- query;
- header.

Exemplo:

```
GET /pedidos/PED-12345?incluirItens=true
```

Evite:

```
GET /pedidos
Content-Type: application/json
```

```
{
```

```
"pedidoId": "PED-12345"  
}
```

Na OpenAPI 3.0, o `requestBody` deve ser utilizado em métodos nos quais o corpo possua semântica HTTP bem definida. Para métodos como GET, HEAD e DELETE, consumidores da especificação podem ignorar o `requestBody`.

GET normalmente não possui body de requisição, mas pode e geralmente possui um **body de resposta**.

22. O objeto `requestBody`

O `requestBody` é mais comum em POST, PUT e PATCH.

Exemplo:

```
post:  
  summary: Criar pedido  
  
  requestBody:  
    required: true  
    description: Dados necessários para criar o pedido.  
    content:  
      application/json:  
        schema:  
          $ref: "#/components/schemas/CriarPedidoRequest"  
  
  responses:  
    "201":  
      description: Pedido criado.
```

Campos

`required`

Informa se o body inteiro é obrigatório.

```
required: true
```

`description`

Explica o conteúdo esperado.

```
description: Dados necessários para criar o pedido.
```

content

Informa os tipos de conteúdo aceitos.

```
content:  
  application/json:
```

Outros exemplos:

```
content:  
  application/xml:
```

```
content:  
  multipart/form-data:
```

```
content:  
  application/octet-stream:
```

schema

Define a estrutura do body.

```
schema:  
  $ref: "#/components/schemas/CriarPedidoRequest"
```

23. O objeto `responses`

Toda operação deve descrever suas respostas possíveis.

```
responses:  
  "200":  
    description: Pedido encontrado.  
  
  "400":  
    description: Parâmetro inválido.  
  
  "401":  
    description: Credencial ausente ou inválida.
```

```
"403":
  description: Consumidor sem autorização.

"404":
  description: Pedido não encontrado.

"500":
  description: Erro interno.
```

Os códigos HTTP devem ser colocados entre aspas para evitar interpretações incorretas no YAML:

```
"200":
```

A especificação determina que cada objeto de resposta tenha uma `description`.

24. Body da resposta

Exemplo de resposta:

```
{
  "pedidoId": "PED-12345",
  "status": "EM_PROCESSAMENTO",
  "dataCriacao": "2026-07-15T14:30:00-03:00"
}
```

Definição:

```
responses:
  "200":
    description: Pedido encontrado com sucesso.

    content:
      application/json:
        schema:
          type: object
          required:
            - pedidoId
            - status
            - dataCriacao
          properties:
            pedidoId:
              type: string
              example: PED-12345

            status:
```

```

    type: string
    enum:
      - RECEBIDO
      - EM_PROCESSAMENTO
      - CONCLUIDO
      - CANCELADO
    example: EM_PROCESSAMENTO

  dataCriacao:
    type: string
    format: date-time
    example: "2026-07-15T14:30:00-03:00"

```

Estrutura mental

```

responses
├── "200"
│   ├── description
│   └── content
│       ├── application/json
│       └── schema
│           ├── type
│           ├── required
│           └── properties

```

25. Cabeçalhos da resposta

A API também pode retornar headers.

```

responses:
  "200":
    description: Pedido encontrado.

    headers:
      X-Correlation-Id:
        description: Identificador de rastreamento.
        schema:
          type: string
          format: uuid

      Cache-Control:
        description: Política de cache da resposta.
        schema:
          type: string

    content:

```



```
application/json:
  schema:
    $ref: "#/components/schemas/Pedido"
```

26. O que é um schema?

Um schema descreve a estrutura de um dado.

Considere o JSON:

```
{
  "nome": "Maria",
  "idade": 35,
  "ativo": true
}
```

Seu schema pode ser:

```
type: object

required:
  - nome
  - idade

properties:
  nome:
    type: string

  idade:
    type: integer

  ativo:
    type: boolean
```

O schema informa:

- quais campos existem;
- o tipo de cada campo;
- quais são obrigatórios;
- quais formatos são esperados;
- quais validações devem ser aplicadas;
- quais valores são permitidos.

27. Tipos de dados

Os tipos básicos do OpenAPI 3.0 são:

Tipo	Exemplo
string	"PED-12345"
integer	10
number	49.90
boolean	true
array	["A", "B"]
object	{"nome": "Maria"}

Esses são os tipos básicos definidos para schemas OpenAPI 3.0.

String

```
type: string
```

Número inteiro

```
type: integer
```

Número decimal

```
type: number  
format: double
```

Booleano

```
type: boolean
```

Array

```
type: array  
items:  
  type: string
```

Objeto

```
type: object
properties:
  nome:
    type: string
```

28. O campo `format`

`format` complementa o tipo.

```
type: string
format: date
```

Exemplo:

```
2026-07-15
```

```
type: string
format: date-time
```

Exemplo:

```
2026-07-15T14:30:00-03:00
```

```
type: string
format: uuid
```

Exemplo:

```
550e8400-e29b-41d4-a716-446655440000
```

Formatos comuns:

Tipo	Formato	Exemplo
<code>string</code>	<code>date</code>	<code>2026-07-15</code>
<code>string</code>	<code>date-time</code>	<code>2026-07-15T14:30:00Z</code>

Tipo	Formato	Exemplo
string	uuid	550e8400-...
string	email	nome@empresa.com.br
string	binary	Arquivo binário
integer	int32	Inteiro de 32 bits
integer	int64	Inteiro de 64 bits
number	float	Ponto flutuante
number	double	Ponto flutuante de maior precisão

O campo `format` é uma indicação adicional para ferramentas e validadores; ferramentas que não reconhecem um formato específico podem considerar apenas o `type`.

29. Campos obrigatórios: três significados diferentes

O nome `required` aparece em contextos diferentes.

29.1 Parâmetro obrigatório

```
parameters:
  - name: pedidoId
    in: path
    required: true
```

Significado:

O consumidor é obrigado a enviar esse parâmetro.

29.2 Body obrigatório

```
requestBody:
  required: true
```

Significado:

O consumidor é obrigado a enviar um body.

29.3 Propriedades obrigatórias dentro de um objeto

```
type: object

required:
  - nome
  - documento

properties:
  nome:
    type: string

  documento:
    type: string

  apelido:
    type: string
```

Significado:

- `nome` é obrigatório;
- `documento` é obrigatório;
- `apelido` é opcional.

Observe que, dentro do schema, `required` é uma **lista com os nomes dos campos obrigatórios**.

Não se escreve:

```
properties:
  nome:
    type: string
    required: true
```

Em OpenAPI 3.0, a forma correta é:

```
required:
  - nome

properties:
  nome:
    type: string
```

30. Obrigatório não significa não nulo

São conceitos diferentes.

Campo obrigatório

Significa que a propriedade deve aparecer no objeto.

```
required:  
  - nome
```

Campo anulável

Significa que o valor pode ser `null`.

Em OpenAPI 3.0:

```
nome:  
  type: string  
  nullable: true
```

Um campo pode ser:

Obrigatório	Anulável	Interpretação
Sim	Não	Deve existir e possuir valor
Sim	Sim	Deve existir, mas pode ser <code>null</code>
Não	Não	Pode ser omitido; se existir, deve possuir valor válido
Não	Sim	Pode ser omitido ou enviado como <code>null</code>

Essa distinção deve ser definida explicitamente durante o refinamento.

31. Validações de string

```
codigo:  
  type: string  
  minLength: 3  
  maxLength: 20  
  pattern: "^PED-[0-9]+$"  
  example: PED-12345
```

Campos

Campo	Significado
<code>minLength</code>	Tamanho mínimo

Campo	Significado
<code>maxLength</code>	Tamanho máximo
<code>pattern</code>	Expressão regular
<code>enum</code>	Lista de valores permitidos
<code>format</code>	Formato conhecido ou convencionado

Enumeração

```
status:
  type: string
  enum:
    - RECEBIDO
    - EM_PROCESSAMENTO
    - CONCLUIDO
    - CANCELADO
```

Isso informa que qualquer outro valor é inválido.

32. Validações numéricas

```
quantidade:
  type: integer
  minimum: 1
  maximum: 100
  example: 2
```

```
valor:
  type: number
  format: double
  minimum: 0
  example: 49.90
```

Campos comuns:

Campo	Significado
<code>minimum</code>	Menor valor permitido
<code>maximum</code>	Maior valor permitido
<code>exclusiveMinimum</code>	Limite inferior exclusivo
<code>exclusiveMaximum</code>	Limite superior exclusivo

Campo	Significado
<code>multipleOf</code>	Valor deve ser múltiplo do número informado

33. Arrays

JSON:

```
{
  "itens": [
    {
      "codigo": "PROD-001",
      "quantidade": 2
    },
    {
      "codigo": "PROD-002",
      "quantidade": 1
    }
  ]
}
```

Schema:

```
itens:
  type: array
  minItems: 1
  maxItems: 100
  items:
    type: object
    required:
      - codigo
      - quantidade
    properties:
      codigo:
        type: string
      quantidade:
        type: integer
        minimum: 1
```

Campos de array

Campo	Significado
<code>items</code>	Schema de cada item
<code>minItems</code>	Quantidade mínima

Campo	Significado
<code>maxItems</code>	Quantidade máxima
<code>uniqueItems</code>	Exige itens únicos

Quando `type` é `array`, o campo `items` descreve o conteúdo de cada posição.

34. Objetos aninhados

JSON:

```
{
  "pedidoId": "PED-12345",
  "cliente": {
    "nome": "Maria",
    "documento": "***.456.789-**"
  }
}
```

Schema:

```
type: object

properties:
  pedidoId:
    type: string

  cliente:
    type: object
    required:
      - nome
    properties:
      nome:
        type: string

      documento:
        type: string
```

35. Os campos `description`, `example` e `default`

Esses campos têm funções diferentes.

description

Explica o significado:

```
description: Número da página solicitada.
```

example

Mostra um exemplo:

```
example: 1
```

default

Informa o valor assumido quando o consumidor não envia o campo:

```
default: 1
```

Exemplo completo:

```
pagina:  
  type: integer  
  minimum: 1  
  default: 1  
  example: 2  
  description: Número da página solicitada.
```

Não use `default` apenas para mostrar um exemplo. Se não existir um comportamento padrão real, use somente `example`.

36. Os campos `readOnly` e `writeOnly`

readOnly

Campo retornado pela API, mas que não deve ser enviado pelo consumidor.

```
dataCriacao:  
  type: string  
  format: date-time  
  readOnly: true
```

writeOnly

Campo que pode ser enviado, mas não deve aparecer na resposta.

```
senha:
  type: string
  format: password
  writeOnly: true
```

A especificação determina que uma mesma propriedade não seja simultaneamente `readOnly: true` e `writeOnly: true`.

37. O objeto components

`components` armazena definições reutilizáveis.

Sem reutilização:

```
responses:
  "200":
    content:
      application/json:
        schema:
          type: object
          properties:
            pedidoId:
              type: string
            status:
              type: string
```

Com reutilização:

```
components:
  schemas:
    Pedido:
      type: object
      properties:
        pedidoId:
          type: string
        status:
          type: string
```

A operação passa a referenciar o componente:

```
responses:
  "200":
    description: Pedido encontrado.
    content:
      application/json:
        schema:
          $ref: "#/components/schemas/Pedido"
```

Componentes possíveis

```
components:
  schemas: {}
  responses: {}
  parameters: {}
  examples: {}
  requestBodies: {}
  headers: {}
  securitySchemes: {}
  links: {}
  callbacks: {}
```

Os mais utilizados são:

- schemas;
- responses;
- parameters;
- requestBodies;
- securitySchemes.

38. O campo \$ref

\$ref significa referência.

```
$ref: "#/components/schemas/Pedido"
```

Leitura:

Use a definição Pedido
que está em components
dentro de schemas
neste mesmo arquivo.

O prefixo #/ representa a raiz do documento.

Exemplo:

```
components:
  schemas:
    Pedido:
      type: object
```

Referência:

```
$ref: "#/components/schemas/Pedido"
```

O uso de `$ref` reduz duplicação e permite que o mesmo modelo seja empregado em múltiplos endpoints.

39. Modelo padronizado de erro

Crie uma estrutura comum para todos os erros.

```
components:
  schemas:
    Erro:
      type: object

      required:
        - code
        - message
        - correlationId
        - timestamp

      properties:
        code:
          type: string
          description: Código estável e tratável do erro.
          example: PEDIDO_NAO_ENCONTRADO

        message:
          type: string
          description: Mensagem compreensível sobre o erro.
          example: O pedido informado não foi encontrado.

        correlationId:
          type: string
          format: uuid
          description: Identificador utilizado para rastreamento.
          example: 550e8400-e29b-41d4-a716-446655440000
```

```

timestamp:
  type: string
  format: date-time
  example: "2026-07-15T14:31:00-03:00"

details:
  type: array
  description: Detalhes adicionais sobre o erro.
  items:
    type: object
    properties:
      field:
        type: string
        example: pedidoId

      reason:
        type: string
        example: Formato inválido.

```

Uso:

```

responses:
  "404":
    description: Pedido não encontrado.
    content:
      application/json:
        schema:
          $ref: "#/components/schemas/Erro"

```

40. Códigos HTTP e códigos de negócio

O código HTTP representa uma categoria técnica.

O código de negócio representa a situação específica.

Exemplo:

```
HTTP/1.1 404 Not Found
```

```

{
  "code": "PEDIDO_NAO_ENCONTRADO",
  "message": "O pedido informado não foi encontrado."
}

```

Tabela recomendada:

HTTP	Código de negócio	Significado
400	PARAMETRO_INVALIDO	Entrada com formato inválido
401	NAO_AUTENTICADO	Credencial ausente ou inválida
403	ACESSO_NEGADO	Consumidor autenticado, mas sem permissão
404	PEDIDO_NAO_ENCONTRADO	Recurso inexistente
409	CONFLITO_DE_ESTADO	Operação incompatível com o estado atual
422	REGRA_DE_NEGOCIO_VIOLADA	Entrada sintaticamente válida, mas rejeitada pela regra
429	LIMITE_EXCEDIDO	Limite de requisições excedido
500	ERRO_INTERNO	Falha não esperada
503	SERVICO_INDISPONIVEL	Serviço temporariamente indisponível

41. Autenticação e autorização

OpenAPI utiliza dois conceitos:

```
securitySchemes = define os mecanismos existentes
security = aplica os mecanismos à API ou operação
```

Os esquemas de segurança do OpenAPI 3.0 incluem autenticação HTTP, API keys, OAuth 2.0 e OpenID Connect. Eles são definidos em `components/securitySchemes` e aplicados pelo campo `security`.

42. Autenticação Bearer/JWT

Definição:

```
components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

Aplicação global:

```
security:
  - BearerAuth: []
```

Isso significa que, por padrão, todos os endpoints exigem:

```
Authorization: Bearer <token>
```

Endpoint público

Uma operação pode remover a exigência global:

```
get:
  summary: Health check
  security: []
  responses:
    "200":
      description: Serviço disponível.
```

43. API Key

```
components:
  securitySchemes:
    ApiKeyAuth:
      type: apiKey
      in: header
      name: X-API-Key
```

Aplicação:

```
security:
  - ApiKeyAuth: []
```

A chamada deverá conter:

```
X-API-Key: valor-da-chave
```

No ambiente AWS, a definição OpenAPI da segurança e a configuração efetiva de autorização do API Gateway devem estar alinhadas. Documentar uma API key no OpenAPI não cria sozinho toda a política de segurança necessária.

44. Exemplo completo de OpenAPI para um GET

```
openapi: 3.0.4

info:
  title: API de Consulta de Pedidos
  description: |
    API utilizada para consultar informações de pedidos.

    Consumidores:
    - Portal de atendimento
    - Aplicativo mobile
    - Sistemas internos autorizados

    Regra principal:
    O consumidor somente pode consultar pedidos pertencentes
    à organização associada à sua credencial.
  version: 1.0.0

contact:
  name: Squad de Pedidos
  email: squad-pedidos@exemplo.com.br

servers:
  - url: https://api.exemplo.com.br/v1
    description: Produção

  - url: https://sandbox-api.exemplo.com.br/v1
    description: Sandbox

tags:
  - name: Pedidos
    description: Operações relacionadas a pedidos.

paths:
  /pedidos/{pedidoId}:
    get:
      tags:
        - Pedidos

      summary: Consultar pedido

      description: |
        Retorna os dados atuais de um pedido.

        Quando `incluirItens` for igual a `true`, a resposta também
        apresentará os itens do pedido.

      operationId: consultarPedido
```

```

parameters:
  - name: pedidoId
    in: path
    required: true
    description: Identificador único do pedido.
    schema:
      type: string
      minLength: 5
      maxLength: 30
      pattern: "^PED-[0-9]+$"
      example: PED-12345

  - name: incluirItens
    in: query
    required: false
    description: Indica se os itens devem ser retornados.
    schema:
      type: boolean
      default: false
      example: true

  - name: X-Correlation-Id
    in: header
    required: false
    description: Identificador utilizado para rastreamento.
    schema:
      type: string
      format: uuid
      example: 550e8400-e29b-41d4-a716-446655440000

responses:
  "200":
    description: Pedido encontrado com sucesso.

    headers:
      X-Correlation-Id:
        description: Identificador de rastreamento da requisição.
        schema:
          type: string
          format: uuid

    content:
      application/json:
        schema:
          $ref: "#/components/schemas/Pedido"

        examples:
          pedidoEmProcessamento:
            summary: Pedido em processamento
            value:
              pedidoId: PED-12345

```

```
status: EM_PROCESSAMENTO
dataCriacao: "2026-07-15T14:30:00-03:00"
cliente:
  nome: Cliente Exemplo
  documento: "***.456.789-**"
itens:
  - codigo: PROD-001
    quantidade: 2
    valorUnitario: 49.90
```

```
"400":
  description: Um ou mais parâmetros são inválidos.
  content:
    application/json:
      schema:
        $ref: "#/components/schemas/Erro"
```

```
"401":
  description: Credencial ausente ou inválida.
  content:
    application/json:
      schema:
        $ref: "#/components/schemas/Erro"
```

```
"403":
  description: Consumidor sem autorização para acessar o pedido.
  content:
    application/json:
      schema:
        $ref: "#/components/schemas/Erro"
```

```
"404":
  description: Pedido não encontrado.
  content:
    application/json:
      schema:
        $ref: "#/components/schemas/Erro"
```

```
"429":
  description: Limite de requisições excedido.
  content:
    application/json:
      schema:
        $ref: "#/components/schemas/Erro"
```

```
"500":
  description: Erro interno.
  content:
    application/json:
      schema:
        $ref: "#/components/schemas/Erro"
```

```

    "503":
      description: Serviço temporariamente indisponível.
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/Erro"

  security:
    - BearerAuth: []

components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

  schemas:
    Pedido:
      type: object

      required:
        - pedidoId
        - status
        - dataCriacao
        - cliente

      properties:
        pedidoId:
          type: string
          description: Identificador único do pedido.
          example: PED-12345

        status:
          type: string
          description: Situação atual do pedido.
          enum:
            - RECEBIDO
            - EM_PROCESSAMENTO
            - CONCLUIDO
            - CANCELADO
          example: EM_PROCESSAMENTO

        dataCriacao:
          type: string
          format: date-time
          description: Data e horário de criação do pedido.
          example: "2026-07-15T14:30:00-03:00"

        cliente:

```

```

    $ref: "#/components/schemas/ClienteResumo"

  itens:
    type: array
    description: |
      Itens do pedido. A propriedade é retornada quando
      o parâmetro `incluirItens` for igual a `true`.
    items:
      $ref: "#/components/schemas/ItemPedido"

ClienteResumo:
  type: object

  required:
    - nome

  properties:
    nome:
      type: string
      description: Nome do cliente.
      example: Cliente Exemplo

    documento:
      type: string
      nullable: true
      description: Documento do cliente parcialmente mascarado.
      example: "***.456.789-**"

ItemPedido:
  type: object

  required:
    - codigo
    - quantidade
    - valorUnitario

  properties:
    codigo:
      type: string
      description: Código do produto.
      example: PROD-001

    quantidade:
      type: integer
      minimum: 1
      description: Quantidade solicitada.
      example: 2

    valorUnitario:
      type: number
      format: double

```

```
    minimum: 0
    description: Valor unitário do item.
    example: 49.90
```

Erro:

```
  type: object
```

required:

- code
- message
- correlationId
- timestamp

properties:

code:

```
  type: string
  description: Código estável do erro.
  example: PEDIDO_NAO_ENCONTRADO
```

message:

```
  type: string
  description: Descrição do erro.
  example: O pedido informado não foi encontrado.
```

correlationId:

```
  type: string
  format: uuid
  description: Identificador utilizado para rastreamento.
  example: 550e8400-e29b-41d4-a716-446655440000
```

timestamp:

```
  type: string
  format: date-time
  description: Data e horário da ocorrência.
  example: "2026-07-15T14:31:00-03:00"
```

details:

```
  type: array
  description: Informações complementares.
  items:
```

```
    type: object
```

properties:

field:

```
  type: string
  example: pedidoId
```

reason:

```
  type: string
  example: O identificador possui formato inválido.
```

45. Como negócio interpreta o mesmo arquivo

Embora o arquivo seja técnico, seus campos podem ser traduzidos para conceitos de negócio.

Campo OpenAPI	Interpretação para negócio
<code>info.title</code>	Nome do serviço
<code>info.description</code>	Objetivo e contexto
<code>tags</code>	Domínios ou grupos de funcionalidades
<code>paths</code>	Serviços disponibilizados
<code>summary</code>	Nome da funcionalidade
<code>description</code>	Regras e comportamento
<code>parameters</code>	Informações que o consumidor deve fornecer
<code>required</code>	Informação obrigatória
<code>enum</code>	Valores de domínio permitidos
<code>responses</code>	Resultados possíveis
<code>schemas</code>	Dicionário dos dados
<code>security</code>	Forma de controle de acesso
<code>examples</code>	Cenários concretos

Essa tradução deve aparecer na apresentação ou no portal de documentação, sem exigir que o usuário de negócio leia diretamente todo o YAML.

46. O que não deve ficar somente no OpenAPI

OpenAPI descreve muito bem o contrato HTTP, mas não substitui toda a documentação funcional e operacional.

Mantenha uma documentação complementar com:

Contexto funcional

- problema de negócio;
- objetivo;
- consumidores;
- responsáveis;
- jornada;
- pré-condições;
- regras de negócio.

Requisitos não funcionais

- latência esperada;
- volume de requisições;
- disponibilidade;
- timeout;
- limite de chamadas;
- política de retry;
- cache;
- retenção de logs;
- classificação da informação;
- requisitos de auditoria.

Operação

- dashboards;
- alarmes;
- procedimentos de contingência;
- runbook;
- responsáveis por suporte;
- dependências;
- política de incidentes.

Governança

- política de versionamento;
- ciclo de aprovação;
- processo de descontinuação;
- compatibilidade;
- histórico de mudanças.

Algumas dessas informações podem ser mencionadas em `description` ou `externalDocs`, mas não devem ser forçadas dentro do contrato quando uma documentação complementar for mais apropriada.

47. Extensões `X -`

OpenAPI permite extensões específicas.

Exemplo genérico:

```
x-owner-team: squad-pedidos
x-data-classification: confidencial
```

Esses campos não fazem parte do padrão principal, mas podem ser interpretados por ferramentas internas.

A AWS utiliza extensões como:

```
x-amazon-apigateway-integration:
```

Exemplo simplificado:

```
paths:
  /pedidos/{pedidoId}:
    get:
      responses:
        "200":
          description: Pedido encontrado.

      x-amazon-apigateway-integration:
        type: AWS_PROXY
        httpMethod: POST
        payloadFormatVersion: "2.0"
        uri: ${lambda_invoke_arn}
```

Atenção

O método exposto para o consumidor pode ser GET:

```
get:
```

Mas o API Gateway pode invocar uma função Lambda internamente usando POST:

```
httpMethod: POST
```

Isso acontece porque a invocação da Lambda é uma comunicação interna entre API Gateway e Lambda, diferente do método HTTP apresentado ao consumidor.

Uma prática possível é manter:

```
openapi.yaml
```

com o contrato funcional e técnico da API, e uma variante de implantação:

```
openapi-aws.yaml
```

com as extensões do API Gateway.

Assim, o contrato permanece legível e os detalhes de infraestrutura não dominam a documentação funcional.

48. Como criar sua primeira especificação

Etapa 1 — Defina o objetivo funcional

Exemplo:

Permitir que sistemas autorizados consultem os dados atuais de um pedido.

Etapa 2 — Identifique o recurso

Recurso: Pedido

Etapa 3 — Defina o endpoint

GET /v1/pedidos/{pedidoId}

Etapa 4 — Liste as entradas

Campo	Local	Obrigatório
pedidoId	Path	Sim
incluirItens	Query	Não
X-Correlation-Id	Header	Não
Token	Authorization	Sim

Etapa 5 — Defina a resposta de sucesso

```
{
  "pedidoId": "PED-12345",
  "status": "EM_PROCESSAMENTO"
}
```

Etapa 6 — Defina o dicionário de campos

Campo	Tipo	Obrigatório	Nulo
pedidoId	String	Sim	Não

Campo	Tipo	Obrigatório	Nulo
status	Enum	Sim	Não
itens	Array	Condicional	Não

Etapa 7 — Defina os erros

400, 401, 403, 404, 429, 500 e 503

Etapa 8 — Defina a autenticação

Exemplo:

Bearer JWT

Etapa 9 — Escreva o YAML

Comece pelo esqueleto:

```
openapi: 3.0.4

info:
  title: API de Consulta de Pedidos
  version: 1.0.0

paths:
  /pedidos/{pedidoId}:
    get:
      summary: Consultar pedido
      responses:
        "200":
          description: Pedido encontrado.
```

Depois adicione progressivamente:

```
parameters
  ↓
schemas
  ↓
errors
  ↓
security
```

↓
examples

Etapa 10 — Valide o arquivo

Abra o documento em um editor compatível, como Swagger Editor.

A ferramenta identifica problemas como:

- indentação incorreta;
- campos desconhecidos;
- referências inválidas;
- ausência de campos obrigatórios;
- tipos incompatíveis;
- estrutura incorreta.

Etapa 11 — Gere a visualização

Utilize Swagger UI ou Redoc para apresentar a documentação.

Negócio não precisa revisar o YAML diretamente. Pode revisar a página gerada.

Etapa 12 — Versione no Git

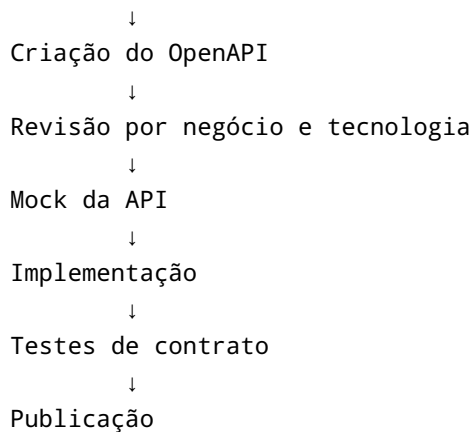
Estrutura sugerida:

```
api-pedidos/  
├── README.md  
├── openapi.yaml  
├── docs/  
│   ├── visao-funcional.md  
│   ├── regras-de-negocio.md  
│   └── codigos-de-erro.md  
└── examples/  
    ├── resposta-sucesso.json  
    └── resposta-erro.json
```

49. Contract First

Uma abordagem recomendada é definir o contrato antes da implementação.

```
Refinamento funcional  
↓  
Desenho do endpoint
```



Essa abordagem é frequentemente chamada de:

API First

ou:

Contract First

O arquivo OpenAPI passa a ser uma fonte de verdade compartilhada.

Negócio valida:

- significado;
- regras;
- informações retornadas;
- cenários;
- erros funcionais.

Tecnologia valida:

- método HTTP;
- URL;
- tipos;
- formatos;
- autenticação;
- códigos HTTP;
- compatibilidade;
- restrições.

50. Versionamento

Existem pelo menos três versões diferentes no projeto.

Versão da especificação OpenAPI

```
openapi: 3.0.4
```

Versão do contrato da API

```
info:  
  version: 1.2.0
```

Versão major no endereço

```
/v1/pedidos
```

Exemplo de alteração compatível:

```
Adicionar um campo opcional na resposta.
```

Pode resultar em:

```
version: 1.1.0
```

Exemplo de alteração incompatível:

```
Remover um campo obrigatório ou alterar seu tipo.
```

Pode exigir:

```
/v2/pedidos
```

As regras exatas devem ser definidas pela governança de APIs da organização.

51. Checklist de refinamento de um endpoint

Identificação

- [] A operação possui nome funcional?
- [] O objetivo está claro?
- [] Os consumidores foram identificados?
- [] O responsável foi definido?

Endpoint

- ☐ O método HTTP foi escolhido?
- ☐ O caminho está definido?
- ☐ O `operationId` é único?
- ☐ A tag foi definida?

Entrada

- ☐ Todos os parâmetros estão documentados?
- ☐ Cada parâmetro informa onde é enviado?
- ☐ Os obrigatórios estão identificados?
- ☐ Os tipos estão definidos?
- ☐ Os formatos estão definidos?
- ☐ Existem exemplos?
- ☐ Existem limites de tamanho?
- ☐ Os valores de domínio estão documentados?

Resposta

- ☐ O body de sucesso está definido?
- ☐ Todos os campos possuem descrição?
- ☐ Os campos obrigatórios estão identificados?
- ☐ A possibilidade de `null` está definida?
- ☐ Arrays informam o schema de seus itens?
- ☐ Datas possuem formato?
- ☐ Valores monetários possuem regra de representação?
- ☐ Há exemplos realistas?

Erros

- ☐ Os erros 400, 401, 403, 404 e 500 foram avaliados?
- ☐ O erro possui estrutura padronizada?
- ☐ Existe um código de negócio estável?
- ☐ Existe correlation ID?
- ☐ Mensagens não expõem detalhes internos?

Segurança

- ☐ A autenticação está definida?
- ☐ A autorização foi descrita?
- ☐ Os dados sensíveis foram identificados?
- ☐ Existe mascaramento quando necessário?
- ☐ A classificação da informação foi definida?

Requisitos não funcionais

- ☐ Latência esperada?
- ☐ Volume estimado?
- ☐ Timeout?

- [] Limite de requisições?
- [] Cache?
- [] Retry?
- [] Disponibilidade?
- [] Logs, métricas e tracing?
- [] Retenção dos logs?
- [] Procedimento de contingência?

52. Modelo recomendado de documentação final

Para atender negócio e tecnologia, publique o material em cinco partes:

1. Visão funcional
2. Catálogo de endpoints
3. OpenAPI
4. Requisitos não funcionais
5. Operação e suporte

Parte 1 — Visão funcional

Por que a API existe?
Quem utiliza?
Qual problema resolve?
Quais regras aplica?

Parte 2 — Catálogo

Método	Endpoint	Finalidade
GET	/v1/pedidos/{pedidoId}	Consultar pedido
GET	/v1/pedidos	Pesquisar pedidos

Parte 3 — Contrato OpenAPI

openapi.yaml

Parte 4 — Requisitos não funcionais

SLA, SL0, latência, capacidade, timeout, cache e segurança.

Parte 5 — Operação

Dashboards, alarmes, runbook, responsáveis e contingência.

53. Resumo final

OpenAPI é uma forma padronizada de escrever o contrato de uma API.

A estrutura principal é:

```
openapi
  Versão do padrão

info
  Identidade da API

servers
  Ambientes e endereços

paths
  Endpoints

operations
  GET, POST, PUT, PATCH e DELETE

parameters
  Entradas em path, query, header ou cookie

requestBody
  Corpo enviado pelo consumidor

responses
  Resultados e códigos HTTP

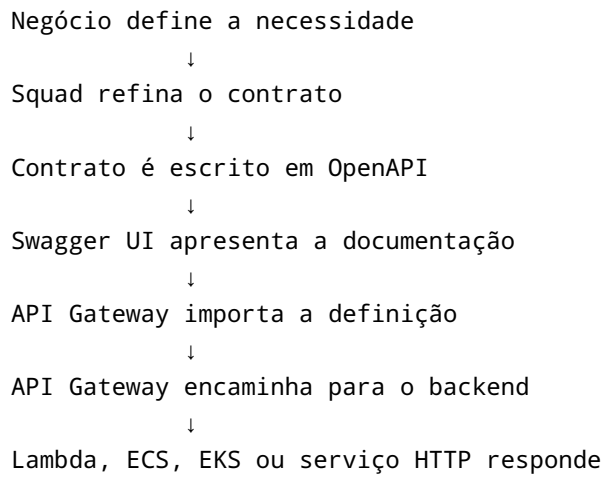
schemas
  Estrutura dos dados

components
  Definições reutilizáveis

securitySchemes
  Mecanismos de autenticação

security
  Aplicação da autenticação
```

A relação completa no ambiente AWS pode ser representada assim:



O OpenAPI deve ser tratado como um contrato versionado e revisado. Ele não substitui toda a documentação funcional, arquitetural e operacional, mas centraliza a definição precisa de como consumidores e provedores se comunicam.